

NotebookLM Source: Guia de Arquitetura SQLite + Event Log (Outbox Pattern)

Este documento detalha o funcionamento da arquitetura baseada em SQLite puro da Semana 15 e o padrão de Log de Eventos.

1. O Padrão Transactional Outbox (Event Log)

Em sistemas orientados a eventos, realizar alterações no banco de dados e disparar chamadas externas de rede (como enviar e-mails ou publicar mensagens no RabbitMQ/Kafka) na mesma transação é um antipadrão. Se a chamada de rede falhar, a transação não poderá ser revertida facilmente.

Para resolver isso de forma simples e robusta, a arquitetura da Semana 15 implementa a persistência de eventos no banco utilizando uma tabela dedicada chamada **event_log**.

Como funciona o fluxo:

1. Uma requisição de escrita (ex: **POST /allocations**) gera um comando **Allocate**.
2. O **Handler** executa a transação através do **Unit of Work**:
 - Salva a alocação na tabela **allocations**.
 - Publica o evento (**Allocated** ou **OutOfStock**) salvando-o na tabela **event_log**.
3. Ambos os registros são salvos **na mesma transação atômica** (**uow.commit()**).
4. Um processo independente (o **Event Consumer**), rodando em segundo plano, consulta periodicamente (polling) a tabela **event_log** para processar e propagar as mensagens.

2. O Consumidor de Eventos (event_consumer.py)

O consumidor funciona em um loop contínuo (polling) buscando eventos novos que ainda não foram processados.

```
def consume():
    last_id = 0 # Controla o último ID lido para não repetir mensagens

    while True:
        conn = get_connection()
        # Busca apenas os novos logs criados desde a última leitura
        rows = conn.execute(
            "SELECT id, channel, event_type, payload FROM event_log WHERE id > ? ORDER BY id",
            (last_id,),
        ).fetchall()
        conn.close()

        for row in rows:
            print(f"Novo evento ID {row['id']}: {row['event_type']} no canal {row['channel']}")
            # Atualiza o ponteiro de leitura
            last_id = row["id"]

        time.sleep(2) # Aguarda 2 segundos antes de consultar novamente
```

3. Peculiaridades do SQLite3 no Python

1. `conn.row_factory = sqlite3.Row`

Por padrão, o conector `sqlite3` retorna as tuplas do banco de dados como tuplas simples (ex: `("batch-001", "CHAIR", 100)`). Acessar dados por índice (`row[0]`, `row[1]`) torna o código ilegível e propenso a erros.

Configurando `conn.row_factory = sqlite3.Row`, o cursor retorna objetos que se comportam como dicionários:

```
conn.row_factory = sqlite3.Row
row = conn.execute("SELECT ref, sku, qty FROM batches").fetchone()
print(row["ref"]) # Acesso direto e legível por nome de coluna
```

2. Controle de Transação no Unit of Work (`unit_of_work.py`)

No SQLite puro, as conexões controlam a transação de forma manual:

- A transação é iniciada automaticamente no primeiro comando SQL de modificação.
- `conn.commit()`: Grava todas as alterações fisicamente no banco.
- `conn.rollback()`: Descarta todas as alterações pendentes caso ocorra algum erro (exceção no bloco `with`).
- `conn.close()`: Fecha a conexão. Deve ser sempre executada no final do bloco do UOW.

4. Reconstrução de Agregados no Repositório (`repository.py`)

No mapeamento imperativo do SQLAlchemy, a ferramenta gerencia o relacionamento automaticamente. No SQLite puro, **você** é responsável por reconstruir o grafo de objetos na mão no método `get_product(sku)`:

```
def get_product(self, sku):
    # 1. Recupera as linhas brutas da tabela 'batches' correspondentes ao SKU
    rows = self.conn.execute("SELECT ref, sku, qty FROM batches WHERE sku = ?", (sku,)).fetchall()
    if not rows:
        return None

    # 2. Instancia o agregado principal (Product)
    product = Product(sku)

    # 3. Itera nos lotes encontrados, busca suas alocações e preenche o agregado
    for row in rows:
        batch = Batch(row["ref"], row["sku"], row["qty"])

        # Busca alocações físicas vinculadas a este lote
        allocations = self.conn.execute("SELECT orderid, sku, qty FROM allocations WHERE batchref = ?", (row["ref"],)).fetchall()
        for alloc in allocations:
            batch.allocations.add(OrderLine(alloc["orderid"], alloc["sku"], alloc["qty"]))

        product.add_batch(batch)

    return product
```

Esse mapeamento manual garante que a lógica de domínio continue sem dependências de infraestrutura.